

K-Waay 批处理接纳中参与方区分条件的形式化分析

作者 A, 作者 B (待补充并核验)

(作者单位、城市、邮编待补充并核验)

基金项目、作者简介及通信作者信息待补充并核验

摘要: K-Waay 的 BatchReceive 规定同一调用中的输入应对应不同参与方。为分析该条件在批处理接纳边界上的语义作用, 构建共享生命周期的三种固定两槽 Tamarin 模型。验证表明: 放宽条件时, 一个精确发送来源可在同一批次支持两次接收事件; 消息级限制虽可保证消息区分和作用域内来源单射性, 仍允许同一参与方的不同消息共同被接受; 参与方级限制保持目标关系且存在有效双接受执行。结果说明, 当前符号抽象中的消息区分不能替代批内参与方区分; 结论限于理想来源匹配和固定两槽模型。

关键词: K-Waay; BatchReceive; 形式化验证; Tamarin; 批处理接纳

Formal Analysis of the Distinct-Party Condition in K-Waay Batch Admission

Author A, Author B (to be completed and verified)

Abstract: K-Waay specifies that the inputs of one BatchReceive call correspond to different parties. To examine the semantic role of this condition at the batch-admission boundary, three fixed two-slot Tamarin models with a shared lifecycle are constructed. Verification shows that, after the condition is relaxed, one exact sender origin can support two receiver-acceptance events in the same batch. A message-level restriction ensures message distinction and scoped exact-origin injectivity, but still admits two different messages from the same party. The party-level restriction preserves the target relation while retaining a reachable valid double-acceptance execution. Thus, under the current symbolic abstraction, message distinction cannot substitute for within-batch party distinction. The conclusion is limited to idealized exact-origin matching and fixed two-slot models.

Key words: K-Waay; BatchReceive; formal verification; Tamarin; batch admission

0 引言

面向量子迁移的异步安全通信需要同时处理密码原语、会话状态和消息组合三个层次的问题。认证密钥交换协议通常先刻画单次发送或单个会话的来源、伙伴关系及密钥安全性, 但接收方在离线后集中处理多条发送方消息时, 还需要决定哪些输入能够被组合到同一次接收操作中。已有安全消息协议分析表明, 从单个会话得到的性质并不会自动覆盖会话管理或组合层的语义[1,2,3]。因此, 批处理输入之间的关系应与单个输入是否具有有效来源分开讨论。

K-Waay 是一种面向量子环境、具有 X3DH 类异步交互方式的可否认认证密钥交换协议[4]。为复用接收方的临时密钥材料, 该协议定义了 BatchReceive: 接收方可在一个调用中处理由多个发送方公钥、预密钥包和协议消息组成的输入向量。K-Waay 完整版本在给出构造后明确假定, 同一次 BatchReceive 调用中的不同输入元素对应不同参与方 (party) (见文献[5], Sec. 5.1, Fig. 10, p. 21)。这一条件原本就是协议输入域的一部分, 本文既不把它视为遗漏, 也不据此声称发现了合规 K-Waay 执行中的协议漏洞。

本文关注该条件在批处理接纳边界上的语义作用。一个输入具有合法的发送来源, 只说明该输入能够与某次发送对应; 它并不说明同一批次中的其他输入来自不同参与方。同一参与方可以在不同发送实例 (sender occurrence) 中产生不同消息, 而相同的公开输入也可能被重复放入两个位置。由此需要区分参与方身份、发送实例、消息、槽位、批次和接收方上下文。消息级去重比较的是消息坐标, 精确来源的单次接受比较的是事件对应关系, 参与方级条件约束的则是多个输入在同一批次中的联合组成。这些概念及单射对应本身已有充分研究[6,7]; 本文的对象限定为它们在 K-Waay BatchReceive 接口所引出

的两槽接纳问题中的关系。

为避免将密码运算与组合关系混在同一比较中, 本文建立三个具有共同生命周期的固定两槽 Tamarin 模型[8]。宽松接纳模型 R 不比较参与方或消息坐标; 消息级模型 M 仅要求两槽消息不同; 参与方级模型 P 直接要求两槽参与方不同, 作为目标条件的正控制。三个模型共享参与方创建、发送实例生成、公开输入收集、批次创建、顺序处理和精确来源匹配, 主要比较变量是收集完成后的接纳谓词。P 所采用的直接比较只是抽象中的一种可执行实现, 并不表示真实协议只能在接收方以同样方式检查该条件。

现有机器验证和独立复核支持以下受限结论。在 R 中, 存在一个精确匹配的发送来源在同一批次上下文内支持两次模型接收事件 ReceiverAccept 的执行, 相应的作用域内精确来源单射性 (scoped exact-origin injectivity) 不成立。在 M 中, 接受消息区分和作用域内精确来源单射性成立, 但仍存在同一参与方通过两个发送实例产生不同消息并在同一批次中各被接受一次的执行。在 P 中, 接受事件之间的参与方区分成立, 同时至少存在一个由不同参与方输入组成并完成处理的有效批次。上述结果说明, 在当前新鲜性、来源匹配和两槽顺序处理规则下, 消息级限制不能直接替代参与方级组合条件; 它们并不构成对所有密码协议的一般定理。

本文的工作集中为三点: 1) 建立共享生命周期的 R、M、P 三种批处理接纳模型, 使不同坐标上的限制能够在相同观察边界下比较; 2) 通过存在性见证和安全性质的机器验证, 区分精确输入重复、同参与方不同消息以及不同参与方正常批次三类行为; 3) 在明确模型假设的前提下说明, 按参与方解释批内输入的接口需要显式维护相应组合关系, 不能仅从消息不同或单个输入具有来源推得该关系。这一结论是有限的接口设计启示, 不规定参与方关系必须由调用方、批次构造器或接收方中的哪一方维护。

本文的直接形式化证据止于模型事件 `ReceiverAccept`。模型使用理想化的精确发送来源事实，未编码完整的签名、KEM、KDF、接收方状态与预密钥绑定、密钥输出、KEY/TEST 查询或应用安装过程；也未建立从 K-Waay 完整执行到该抽象的精细化关系。机器分析固定为两个槽位，不能据此声称任意批次大小下的性质，也不能将模型见证提升为密钥泄露、完整认证失败、部署漏洞或实际批次构造器攻击。后文首先界定 K-Waay 接口、身份坐标和分析能力，再给出三个模型的共同规则、接口语义和验证性质。

1 K-Waay 批处理接纳及参与方区分问题

1.1 K-Waay 的 BatchReceive 接口

K-Waay 将可否认认证密钥交换抽象为 `KeyGen`、`Init`、`Send` 和 `BatchReceive` 四个算法（见文献[5]，Sec. 4.1）。参与方 P_i 作为接收方时，先执行 `Init` 获得接收方会话状态 st_i 和预密钥包 $prek_i$ ；一个或多个发送方 P_j 可针对该预密钥包执行 `Send`，产生会话密钥和协议消息。接收方恢复在线后，以自己的长期秘密密钥 sk_i 、同一接收方状态 st_i 以及包含一个或多个发送方相关分量的向量 S 调用 `BatchReceive`。按照原论文第 4.1 节的语法，可将该接口写为

$$\{k_j\}_{j=1}^d \leftarrow \text{BatchReceive}(sk_i, st_i, S = ((pk_{j_\ell}, prek_{j_\ell}, mj_\ell))_{\ell=1}^d), d \geq 1. \quad (1)$$

其中， pk_{j_ℓ} 、 $prek_{j_\ell}$ 和 mj_ℓ 分别表示声称的发送方 P_{j_ℓ} 的长期公钥、预密钥包和协议消息。原论文称输入为向量；本文用 ℓ 表示向量位置，用 j_ℓ 表示该位置所标识的对端参与方，避免把位置编号与参与方编号混为一谈。

K-Waay 完整构造的图 10 进一步给出了接收处理过程（见文献[5]，Sec. 5.1, Fig. 10）。接收方状态在该构造中包含临时 `split-KEM` 和 `ephemeral-KEM` 的秘密材料及本方预密钥包。算法逐个处理由下标 j 标识的输入分量，验证预密钥签名，执行长期 KEM、临时 KEM 和 `split-KEM` 的解封装，并把双方身份、公钥、预密钥包及消息组合进原协议的 `transcript sid`，随后导出对应的 k_j 。如果某个签名验证失败，该分量可得到 $\perp$ 并继续处理；如果 `split-KEM` 解封失败，则设置批次失败标志，最终返回全 $\perp$ 向量；否则返回由各分量结果组成的 $\{k_j\}_j$ 。本文只保留这些与参与方索引、共享接收方状态和批处理结果有关的接口事实，不复述完整密码学构造。

在图 10 之后，原文明确说明：对一次给定的 `BatchReceive` 调用， S 的每个元素对应不同参与方（见文献[5]，Sec. 5.1, p. 21）。该条件约束的是同一调用内部输入分量之间的关系，并不禁止某个参与方在不同会话、不同批次或不同时间产生多条消息。本文将它记为批内参与方区分条件，而不把它解释为新发现的协议要求。

原论文的安全模型也解释了参与方下标为何与接口语义有关。对于接收方会话， pid 、 sid 和 k 均可作为向量；启动接收方会话时，挑战者记录对端参与方索引向量，执行查询提供按这些索引组织的输入，`KEY(i,s,j)` 和 `TEST(i,s,j)` 则以 j 选择接收方会话中的 k_j （见文献[5]，Secs. 4.2.1–4.2.3, pp. 16–18）。因此，在原文规定的合法输入域内，同一下标 j 把声称的对端参与方、输入分量和输出密钥分量联系起来。

这一记号不能被扩展为原文没有给出的语义。K-Waay 没有把“槽位”定义为独立的协议对象，也没有说明同一参与方下标在一次调用中重复出现时，两个位置应如何对应到 k_j 或安全游戏查询。由于不同参与方条件已经排除该输入，重复 j 下的输出选择和 `KEY/TEST` 行为在原论文中并未建立。本文仅把按参与方索引的接口作为研究参与方关系的动机，不从当前模型推出任何 `KEY/TEST` 失败。

为描述输入被组合后是否进入模型处理路径，本文使用“批处理接纳”（`batch admission`）一词。它是分析层边界，不是

K-Waay 原规范中新增加的算法。原条件可以由调用方、批次构造器、接收方接纳层或其他可信组件维护；原论文的自然语言条件本身没有唯一规定其实现位置。

K-Waay源接口（一次BatchReceive调用）

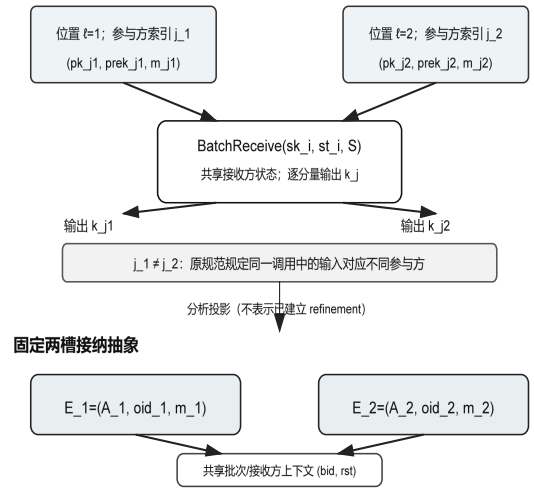


图 1 K-Waay BatchReceive 输入条目、索引输出与参与方投影关系

注：依据 K-Waay 接口事实与本文抽象人工整理；源接口到符号模型之间尚未建立精化证明。

1.2 身份坐标与批内参与方区分

K-Waay 原协议的 `sid` 是由参与方身份、公钥、预密钥包和消息拼接形成并输入 KDF 的 `transcript` 标识。当前 Tamarin 模型中的字段 `sid` 则是每次 `SendMessage` 规则触发时生成的独立新鲜原子，不包含上述结构。为避免两种含义混淆，本文在叙述和公式中将模型字段统一记为 `oid`，即

$$oid \equiv sid_{mo} del. \quad (2)$$

这一约定只是论文叙述层的重命名；三个 `.sphy` 源文件仍使用变量名 `sid`，模型、事实和已验证 `lemma` 均未因此改变。

本文把一个候选接纳条目抽象为

$$El = (A_\ell, oid_\ell, m_\ell), \quad (3)$$

其中 A_ℓ 是模型中的参与方坐标， oid_ℓ 标识一次发送实例， m_ℓ 是该实例生成的符号消息， ℓ 是条目在批次中的槽位。这个三元组不是 K-Waay 的线上输入格式，也不把 A 等同于账户、公钥字节串、预密钥记录或实现数据库中的身份对象。各坐标的作用如下。

坐标说明：参与方 A ——条目投影到的模型协议主体；不自动等同于公钥编码、账户或预密钥。**发送实例 oid** ——一次 `SendMessage` 规则触发的模型内来源标识；不等同于 K-Waay `transcript sid`。**消息 m** ——该发送实例产生并公开的符号消息坐标；不表示具体 KEM 密文结构或真实内容相等规则。**槽位 ℓ** ——条目在当前两槽模型中的位置；位置不同不推出参与方或消息不同。**批次 B / bid** ——一组条目及其模型内批次标识；不要求参与方在全局只出现一次。**接收方上下文 (receiver context) rst** ——使两个处理事件归入同一模型上下文的新鲜标识；不重建 st_i 、预密钥或接收方密码状态。

令有限有序批次 $B = (E_1, \dots, E_n)$ ， $party(E_\ell)$ 表示条目的参与方投影。本文定义

$$\text{DistinctPartyPerBatch}(B) \triangleq \forall 1 \leq p < q \leq n, party(E_p) \neq party(E_q). \quad (4)$$

该定义描述任意有限批次上的目标语义：同一批次中任意两个不同位置都投影到不同参与方。它不要求参与方在不同批

次或不同会话间全局唯一，也不包含密钥机密性、认证或活性结论。当前机器验证固定 $n = 2$ ，此时目标退化为 $A_1 \neq A_2$ 。两槽能够表示最小的违反对，但两槽结果不是任意 n 的机器证明。

发送实例和消息的区分不能替代参与方区分。同一参与方可以产生两个不同的模型条目

$$E_1 = (A, \text{oid}_1, m_1), E_2 = (A, \text{oid}_2, m_2), \text{oid}_1 \neq \text{oid}_2, m_1 \neq m_2, \quad (5)$$

因而 $m_1 \neq m_2$ 并不推出 $A_1 \neq A_2$ 。另一方面，把同一条目放入两个槽位得到 $E_1 = E_2 = (A, \text{oid}, m)$ ，这是精确重复输入；它同时重复参与方、发送实例和消息。前一种情形考察两个独立来源的联合组成，后一种情形考察一个精确来源是否能支持多个接受事件。二者必须分开分析。

这种区分也适用于来源证据。若一个接受条目与 $\text{Send}(A, \text{oid}, m)$ 精确对应，可以说明该条目在模型中具有先前发送来源；但即使批内每个条目都各自具有来源，也不能由此推出参与方投影在整个批次中是单射的。批内参与方关系是组合谓词，不是单个条目来源对应性的同义表达。

1.3 研究问题、对手能力与证据终点

本文考虑符号化的批次组合对手（batch-composition adversary）。发送规则把三元组 $\langle A, \text{oid}, m \rangle$ 暴露到 Dolev-Yao 网络后，对手可以保留和重放已知三元组、在已知条目之间作选择、把它们安排到两个槽位，并把形成的候选组合交给接纳规则。该能力位于“已有候选条目”与“组成一个待接纳批次”之间；它不允许对手创建诚实参与方的内部状态，也不意味着真实部署中的网络攻击者一定控制批次构造器。

在此能力下，本文区分两类输入。精确重复输入把同一 $\langle A, \text{oid}, m \rangle$ 交付两次，依赖公开条目的重放；同参与方不同消息输入则选择同一 A 的两个真实发送来源 $\langle A, \text{oid}_1, m_1 \rangle$ 和 $\langle A, \text{oid}_2, m_2 \rangle$ ，无需重复同一消息。两类输入均不满足 K-Waay 原文的批内不同参与方条件，因此它们用于分析放宽接纳后的模型行为，而不是合规 K-Waay 输入域内的攻击轨迹。

对已接受条目，模型要求完整三元组与持久来源事实精确匹配。该理想化匹配排除了仅凭相同参与方坐标为任意消息伪造来源，但它没有执行 K-Waay 的签名验证、解封装、KDF 或接收方状态检查。对手能够在接纳前组合可推导的值；若被接纳条目没有精确来源，后续处理可能停滞。模型没有把这种停

滞解释为原协议的某一种失败返回。

基于上述边界，本文回答两个问题：第一，取消参与方级限制后，在共同两槽生命周期中能否到达精确来源被重复接受的行为；第二，仅约束消息坐标能否保证批内参与方区分，以及直接维护参与方关系时目标性质和有效执行是否同时存在。参与方级模型作为正控制，用来确认目标关系在相同生命周期中的保持和非空性，不被包装成独立的密码学发现。

形式化观察终点是模型动作 $\text{ReceiverAccept}(A, \text{oid}, m, \text{bid}, \text{rst})$ 。它表示某个条目在精确来源事实支持下完成一次抽象处理，不等于 K-Waay 安全模型中的完整会话接受，也不包含 k_j 、KEY/TEST、应用安装或部署状态。本文没有真实批次构造器的攻击证据，也不对原协议在重复参与方扩展域中的输出选择作出补充定义。第 2 章在这一范围内给出 source-to-model 映射和三种接纳语义。

2 批处理接纳机制的符号建模

2.1 源接口到符号模型的映射

本文模型不重建完整 K-Waay，而是隔离两个发送方相关条目在同一接收方上下文中被组合和接纳时的参与方关系。建模目标是让参与方、发送实例、消息、批次和接收方上下文保持可观察，同时在共同生命周期上只改变接纳谓词。省略密码运算有助于控制比较变量，但不能据此断言被省略的运算不会影响真实接收行为。

原协议和模型使用了同名但含义不同的 sid 。K-Waay 图 10 中的 sid 由双方身份、公钥、预密钥包和协议消息拼接得到，并进入 KDF（见文献[5]，Sec. 5.1, Fig. 10）；三个 Tamarin 模型中的 sid 则由 $\text{Fr}(\sim \text{sid})$ 直接生成，只标识一次模型发送实例。本文统一采用

$$\text{oid} \equiv \text{sid}_{\text{model}}, \quad (6)$$

并在论文公式中写 oid 。模型源码、lemma 名称和原始运行输出仍保留 sid ，这一记号替换不表示新增变量或模型修改。因此，源码事实 $!\text{Sent}(A, \text{sid}, m)$ 在本文叙述中写作 $!\text{Sent}(A, \text{oid}, m)$ 。

表 1 给出源接口到符号模型的映射。表中的“保留”只说明本文观察了相应关系，不表示已经证明二者之间的 simulation 或 refinement。

表 1 K-Waay 源接口到接纳模型的映射

Table 1 Mapping from the K-Waay Interface to the Admission Model			
K-Waay 对象	模型表示	保留的关系	未建模或未经证明的部分
输入分量 $(pk_j, \text{pre}k_j, m_j)$	$E = (A, \text{oid}, m)$	一个条目的参与方投影、发送实例和消息坐标	公钥/参与方解析、预密钥内容、线上消息结构
一次 Send	$\text{Send}(A, \text{oid}, m)$ 与公开三元组	一次模型发送来源及其公开可组合输入	Send 的签名、封装、接收方公钥和预密钥绑定
同一 BatchReceive 调用	共享 (bid, rst)	两槽属于同一模型批次和接收方上下文	真实 st_i 的密码状态、预密钥生命周期和调度
原文不同参与方条件	$A_1 \neq A_2$	两槽目标关系	任意批大小的机器证明、具体执行组件
逐分量处理和 k_j	$\text{ReceiverAccept}(A, \text{oid}, m, \text{bid}, \text{rst})$	一个精确来源支持一次模型内处理事件	k_j 、会话状态、KEY/TEST、应用安装
批次失败和分量 $\perp$	无直接对应；M/P 另有抽象 $\text{Reject}(\text{bid}, \text{rst})$ 分支	不保留源协议的密码失败语义；Reject 仅表示消息或参与方接纳条件不满足	K-Waay 的签名失败、split-KEM batch-wide failure 及返回向量与模型 Reject 之间未建立映射，二者不能视为同一安全事件

模型中的持久事实 $!\text{Sent}(A, \text{oid}, m)$ 是理想化的精确发送来源事实。SendMessage 产生该事实，后续处理规则只有在完整三元组 (A, oid, m) 一致时才能触发 ReceiverAccept。它由规则直接提供来源对应性，不是 K-Waay 签名验证、解封装或认证过程的实现。特别地，模型没有编码 signature、KEM、KDF、receiver/prekey binding、原协议 sid 的构造、密钥输出、KEY/TEST 查询或应用安装。

这种抽象既可能允许不能提升为真实协议执行的符号组合，也可能排除完整协议中的其他行为。被接纳的条目如果缺少精确 $!\text{Sent}$ 事实，处理路径可以停滞；模型不会把停滞自动转换成 K-Waay 的某个失败返回。因此，本文不把该抽象称为原协议的天然保守近似，也不声称已经建立从 K-Waay 到 Tamarin 模型的完整精化定理。其结论只适用于下述接纳规则、来源假设和观察事件。

2.2 共同生命周期和两槽范围

三个模型共享相同的参与方创建、发送、批次创建、输入收集和顺序处理规则。共同生命周期可概括为

创建参与方 → 产生发送实例并公开条目 →
收集两槽输入 → 接纳或拒绝 → 顺序处理 →

ReceiverAccept. (7)

首先, CreateParty 生成新鲜参与方原子 A 并保存持久事实 $!Party(A)$ 。该事实可重复使用, 因此同一参与方能够触发多个 SendMessage。每次发送规则从 Fr 分别生成新鲜 oid 和 m , 记录动作

Send(A, oid, m), (8)

向网络公开 $\langle A, oid, m \rangle$, 并保存精确来源事实 $!Sent(A, oid, m)$ 。这里的新鲜性是模型构造: 不同发送规则实例产生不同的 oid 和 m 。它不是关于真实消息编码、碰撞概率、重传或原协议 transcript sid 的结论。

随后, CreateBatch 生成新鲜批次标识 bid 和接收方上下文标识 rst , 并打开第一个槽位。CollectSlot1 从网络接收 $E_1 = (A_1, oid_1, m_1)$, 线性状态推进到第二槽; CollectSlot2 接收 $E_2 = (A_2, oid_2, m_2)$, 形成

Collected ($bid, rst, A_1, oid_1, m_1, A_2, oid_2, m_2$). (9)

bid 仅标识本次模型批次, rst 仅把两个槽位置于同一抽象

表 2 三种两槽接纳模型的语义比较

Table 2 Admission Semantics of the Three Two-Slot Models

模型	接纳条件	拒绝分支	分析角色
R: 宽松接纳 (relaxed admission)	不要求 $A_1 \neq A_2$ 或 $m_1 \neq m_2$	无收集后拒绝分支	移除目标不等关系的基线
M: 消息级接纳 (message-level admission)	记录 $Neq(m_1, m_2)$, 全局限制排除 $Neq(x, x)$	消息相同时可触发 RejectRepeatedMessage	比较消息坐标的辅助控制
P: 参与方级接纳 (party-level admission)	记录 $Neq(A_1, A_2)$, 全局限制排除 $Neq(x, x)$	参与方相同时可触发 RejectRepeatedParty	直接维护两槽目标关系的正控制

R 的 AdmitRelaxedBatch 对参与方和消息均无不等判断, 因而接纳规则本身不排除精确重复条目或同参与方不同消息条目。是否存在完成后续处理并满足特定事件约束的执行, 仍需由存在性性质验证, 不能仅从“没有限制条件”推定完整执行轨迹。

M 的接纳分支比较 m_1 和 m_2 , 但不比较 A_1 和 A_2 。在当前模型中, 每次发送产生 fresh m , 因此消息比较对精确来源重复具有较强区分能力; 真实消息的规范化、重复内容、碰撞和跨批去重均未建模。M 是为检验坐标替代关系而设置的辅助模型, 不代表已发现 K-Waay 的实际去重实现或提出具体修复方案。

P 的接纳分支直接比较 A_1 和 A_2 , 是 DistinctPartyPerBatch 在两槽抽象中的一个可执行实现。该模型用于验证目标关系在共同处理路径上的保持以及有效批次的非空性 (non-vacuity)。直接比较写入接纳条件后, 相关安全性质具有正控制性质; 它不能被描述为从完整密码运算中自动发现的定理。真实系统也可以由调用方、批次构造器或其他可信组件预先维持同一关系, P 并不规定唯一实现位置。

2.4 验证性质与证据口径

模型通过动作事实区分安全性质、存在性见证和拒绝分支可达性。以下公式均使用本文记号 oid : .spthy 源文件中的对应变量名仍为 sid 。记 $X@t$ 表示动作 X 在执行轨迹 (trace) 的时间点 t 发生。

1) 发送来源对应性 (sender-origin correspondence)。对任意接受事件, 要求存在更早的完整三元组匹配发送:

$\forall A, oid, m, bid, rst, r. ReceiverAccept(A, oid, m, bid, rst)@r \Rightarrow \exists s. Send(A, oid, m)@s \wedge s < r.$ (11)

该性质对应三个模型中的 receiver_accept_has_send。由于

接收方上下文。二者均由 Fr 生成, 但 rst 不包含 K-Waay 的 receiver state、长期密钥、临时秘密或预密钥。模型也没有显式接收方参与方对象。

收集完成后, 所选模型执行相应接纳规则。接纳路径记录 BatchReceive(bid, rst) 并进入第一槽处理状态。ProcessSlot1 只有在存在 $!Sent(A_1, oid_1, m_1)$ 时才产生

ReceiverAccept ($A_1, oid_1, m_1, bid, rst$) (10)

并打开第二槽处理状态; ProcessSlot2 以相同方式检查第二个精确来源, 产生第二个 ReceiverAccept 后到达 BatchComplete(bid, rst)。两个处理状态是线性的, 因而处理顺序固定; $!Sent$ 是持久事实, 若同一精确条目被收集两次, 它可以被两个处理规则重复读取。ReceiverAccept 不显式携带槽位编号, 两个事件由不同时间点和线性处理阶段区分。

模型把批次大小固定为两个槽位。DistinctPartyPerBatch 是成对关系, 较大批次中的任何违反都至少包含一对参与方投影相同的位置, 因此两槽足以表达最小违反形状。然而, 较大批次可能具有额外的处理、失败和组合交互; 当前模型没有验证这些行为, 也没有给出从两槽结果归纳到任意批次大小的证明。

2.3 三种接纳语义

三个模型文件在共同生命周期上采用不同的收集后接纳谓词。M 和 P 还分别增加对应的拒绝规则与 Neq 动作; 因此, “主要比较变量是接纳谓词”不等于三个源文件只相差一行。

处理规则直接要求 $!Sent(A, oid, m)$, 它表达的是理想来源事实在处理事件中的保持, 不是完整协议认证或双方 agreement。

2) 接受事件上的参与方区分与消息区分。对一条 trace τ , 定义

$P\tau$

$\triangleq \forall A_1, A_2, oid_1, oid_2, m_1, m_2, bid, rst, r_1, r_2. RA(A_1, oid_1, m_1, bid, rst)@r_1 \wedge RA(A_2, oid_2, m_2, bid, rst)@r_2$

$\wedge r_1 \neq r_2 \Rightarrow A_1 \neq A_2.$ (12)

$M\tau$

$\triangleq \forall A_1, A_2, oid_1, oid_2, m_1, m_2, bid, rst, r_1, r_2. RA(A_1, oid_1, m_1, bid, rst)@r_1 \wedge RA(A_2, oid_2, m_2, bid, rst)@r_2$

$\wedge r_1 \neq r_2 \Rightarrow m_1 \neq m_2.$ (13)

其中 $RA(\cdot)$ 是 ReceiverAccept 的缩写, 两个事件共享同一 (bid, rst)。 $P\tau$ 对应 P 的 accepted_batch_has_distinct_parties, $M\tau$ 对应 M 的 accepted_batch_has_distinct_messages。

3) 作用域内精确来源单射性 (scoped exact-origin injectivity)。记 $I\tau$ 为

$\forall A, oid, m, bid, rst, s, r_1, r_2.; Send(A, oid, m)@s$

$\wedge RA(A, oid, m, bid, rst)@r_1 \wedge RA(A, oid, m, bid, rst)@r_2$

$\wedge s < r_1 \wedge s < r_2 \Rightarrow r_1 = r_2.$ (14)

它只约束同一批次和接收方上下文中的同一精确来源, 不排除跨批次使用, 也不是标准 AKE 的全局 injective agreement。

在共同的消息新鲜性、精确来源对应和两槽处理规则下,

三个执行轨迹谓词之间满足人工推导关系

$$(P\tau \Rightarrow M\tau) \wedge (M\tau \Leftrightarrow I\tau), \quad (15)$$

而消息级模型中的机器见证给出

$$M\tau \not\Rightarrow P\tau. \quad (16)$$

这些关系只适用于当前模型规则和假设。 $P\tau \Rightarrow M\tau$ 及 $M\tau \Leftrightarrow I\tau$ 是对规则、新鲜性和来源性质的人工推导，不是新增 Tamarin 引理； $M\tau \not\Rightarrow P\tau$ 的反例由 M 模型的 `same_party_different_messages_batch_exists` 见证。详细推导和机器/人工证据分界将在第 3 章分析，审计依据另行记录。

4) 存在性与拒绝可达性。`one_send_two_accepts_exists`、`same_party_different_messages_batch_exists` 和 `distinct_party_batch_exists` 等存在性性质用于确认代表性组合能够形成完整观察轨迹，并为相应安全结论提供反例或非空性依据。M/P 的拒绝存在性只证明抽象拒绝分支存在某条可达轨迹。当前引理中的前置 `Send` 没有与 `Reject(bid,rst)` 所对应的被拒条目绑定，因而不能写成“特定合法发送输入已被证明遭到拒绝”，也不能推出所有无效批次最终都会拒绝的活性结论。

三个未修改模型已有独立复核：在 Tamarin 1.12.0、Maude 3.5.1 环境中重新完成解析与证明，14 个模型—引理实例的状态和证明步数均与原记录一致，其中 13 个得到验证、1 个被反例否定；原始标准输出、错误输出和导出图已经保留。该复核确认当前抽象结果可重复，但不恢复早期历史运行的来源记录，也不补足从 K-Waay 源协议到模型的精化关系。完整结果表和各见证的解释留到第 3 章。

3 形式化验证与结果分析

本章依据三个未修改的两槽 Tamarin 模型及 2026 年 9 月 16 日的独立复核证据分析接纳行为。存在性性质用于给出至少一条满足公式的可达执行，全称性质用于约束相应模型的全部执行轨迹；二者承担的证据作用不同。除特别说明外，正文继续使用 $oid \equiv sid_{mo}d_{cl}$ ，并把直接观察终点限定为 `ReceiverAccept`。

3.1 宽松接纳模型中的精确来源重复接受

宽松接纳模型 R 在收集两个条目后直接触发 `AdmitRelaxedBatch`，不比较参与方、发送实例或消息坐标。`normal_relaxed_batch_exists` 以 10 步得到验证，表明该生命周期至少可以到达一次 `BatchReceive` 和随后的 `ReceiverAccept`。这一结果只排除“模型完全不能完成接纳处理”的空模型解释，不提供参与方区分证据。

核心存在性性质 `one_send_two_accepts_exists` 以 13 步得到验证。其见证满足

$$\begin{aligned} & \text{Send}(A, oid, m)@s, \\ & \text{BatchReceive}(bid, rst)@b, \\ & \text{RA}(A, oid, m, bid, rst)@r_1, \\ & \text{RA}(A, oid, m, bid, rst)@r_2, \\ & s < b < r_1 < r_2, \end{aligned} \quad (17)$$

并附加同一精确三元组的发送唯一性条件

$$\forall s_2. \text{Send}(A, oid, m)@s_2 \Rightarrow s_2 = s. \quad (18)$$

这里的唯一性只针对与 (A, oid, m) 完全匹配的 `Send`。它不能被解释为整条执行中只有一次发送；独立复核导出的 graph 确实还包含与见证变量无关的其他 `Send`，但这些事件不改变公式所绑定的精确来源。见证中的同一公开三元组被 `CollectSlot1` 和 `CollectSlot2` 分别收集，随后两个处理规则在同一 (bid, rst) 下复用持久事实 `!Sent(A, oid, m)`，因而在不同时间点产生两个参数完全相同的 `ReceiverAccept`。

与该存在性结果对应，R 中的全称性质 `receiver_accept_injective` 以 13 步被反例否定。该性质要求：如果同一 (A, oid, m) 的发送来源先于同一批次和接收方上下文中的两个匹配接受事件，则两次接受必须发生在同一时间点。上述见证满足其前件，却有 $r_1 < r_2$ ，因此给出直接反例。与此同时，`receiver_accept_has_send` 以 8 步得到验证，说明反例没有依赖无来源条目；两个接受事件共享的是一个理想化的精确来源。

被否定的性质是本文定义的作用域内精确来源单射性，而不是标准 AKE 中的 `injective agreement`。该执行也不属于 K-Waay 原文规定的不同参与方输入域：它只说明，在取消坐标不等限制的当前两槽抽象中，一个公开条目可以占据两个槽并到达两个模型接受事件，不能据此声称完整 K-Waay 存在真实重放攻击或认证失效。

同一精确来源在宽松接纳模型中的两次接受

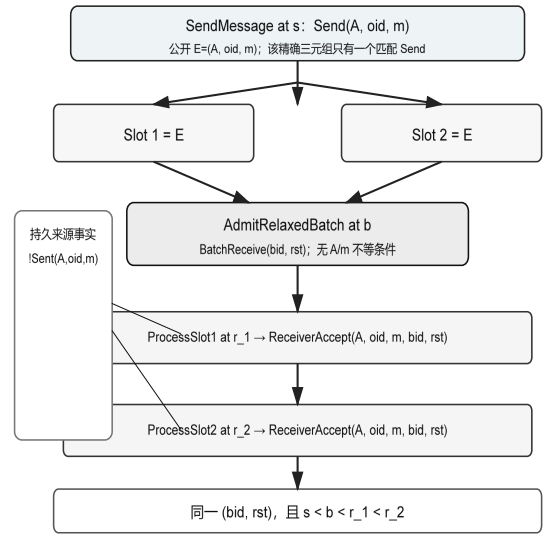


图 2 宽松接纳模型中的精确来源重复接受执行

Fig. 2 Exact-Origin Duplicate-Acceptance Execution in the Relaxed Model

注：根据 Tamarin 见证人工整理，不是 prover 直接导出的原始攻击图。

3.2 消息级接纳模型的限制与反例

消息级接纳模型 M 在接纳分支记录 $\text{Neq}(m_1, m_2)$ ，并由全局限制排除 $\text{Neq}(x, x)$ ，因而接纳条件为

$$m_1 \neq m_2. \quad (19)$$

`repeated_message_rejection_exists` 以 4 步得到验证，只说明消息相等时抽象拒绝分支存在一条可达执行。该 lemma 中的前置 `Send` 没有与 `Reject(bid,rst)` 对应的被拒条目绑定，因此不能据此断言某个特定合法发送输入已被拒绝，也不能推出所有等消息批次最终都会进入拒绝分支。

对接受路径，`accepted_batch_has_distinct_messages` 以 31 步得到验证：同一 (bid, rst) 中两个不同时间点的 `ReceiverAccept` 必须具有不同消息坐标。`receiver_accept_injective` 以 33 步得到验证，表明同一精确来源不能在该上下文中支持两个不同接受事件；`receiver_accept_has_send` 仍以 8 步得到验证。由此，R 中“同一三元组占据两个槽”的精确重复形状不能在 M 的接受路径上重现。

消息级限制并未维护参与方关系。`same_party_different_messages_batch_exists` 以 16 步得到验证，其见证由同一参与方 A 的两个发送实例组成：

$$E_1 = (A, \text{oid}_1, m_1), E_2 = (A, \text{oid}_2, m_2), \quad (20)$$

其中

$$\text{oid}_1 \neq \text{oid}_2, m_1 \neq m_2. \quad (21)$$

机器公式同时绑定两个发送和两个接受:

$\text{Send}(A, \text{oid}_1, m_1)@s_1, \text{Send}(A, \text{oid}_2, m_2)@s_2,$

$\text{BatchReceive}(\text{bid}, \text{rst})@b,$

$\text{RA}(A, \text{oid}_1, m_1, \text{bid}, \text{rst})@r_1, \text{RA}(A, \text{oid}_2, m_2, \text{bid}, \text{rst})@r_2,$

$$s_1 < b, s_2 < b, b < r_1 < r_2. \quad (22)$$

因此, 两次接受分别具有自己的精确匹配来源, 见证没有制造 origin mismatch, 也没有把一个发送错误归属给另一参与方。它展示的是: 在消息坐标已经区分、精确来源单射性已经成立的情况下, 两个条目的参与方投影仍可相同。换言之, M 在其所约束的坐标上有效排除了精确消息重复, 但实际模型行为仍给出 $M\tau$ 不蕴含 $P\tau$ 的反例。

这一结果不能扩展为身份错误归属、未知密钥共享(UKS)、misbinding 或密钥泄露。模型没有对端密钥、会话密钥、KEY/TEST 或应用安装事件; 见证也不在 K-Waay 原文规定的不同参与方合法输入域内。它的证据价值在于说明, message-coordinate control 与批内参与方区分观察的是不同关系。

3.3 参与方级接纳模型的性质验证

参与方级接纳模型 P 把接纳条件直接设为

$$A_1 \neq A_2. \quad (23)$$

AdmitDistinctParties 记录 $\text{Neq}(A_1, A_2)$, 全局不等限制排除参与方相同的接纳分支。相应的全称性质 accepted_batch_has_distinct_parties 以 31 步得到验证: 同一 (bid, rst) 中两个不同 ReceiverAccept 的参与方坐标必不相同。这一结论说明目标关系在当前两槽处理路径上得到保持; 由于接纳规则直接维护同一关系, 它是正控制结果, 而不是从完整

密码运算中发现的新安全定理。

安全性没有依靠拒绝全部输入获得空真。存在性性质 distinct_party_batch_exists 以 17 步得到验证, 给出两个不同参与方及其各自精确来源在一个批次中完成双接受的执行:

$\text{Send}(A_1, \text{oid}_1, m_1)@s_1, \text{Send}(A_2, \text{oid}_2, m_2)@s_2,$

$A_1 \neq A_2,$

$\text{BatchReceive}(\text{bid}, \text{rst})@b,$

$\text{RA}(A_1, \text{oid}_1, m_1, \text{bid}, \text{rst})@r_1, \text{RA}(A_2, \text{oid}_2, m_2, \text{bid}, \text{rst})@r_2,$

$$s_1 < b, s_2 < b, b < r_1 < r_2. \quad (24)$$

该见证只建立非空性, 即至少存在一个有效的双接受执行; 它不证明所有满足参与方区分的候选批次最终都会完成。 P 中 receiver_accept_has_send 和 receiver_accept_injective 分别以 8 步和 33 步得到验证, 为同一处理路径提供模型内来源对应和精确来源单射性, 但不增加协议层认证或活性结论。

same_party_rejection_exists 以 5 步得到验证, 说明参与方相同时抽象拒绝分支可达。与 M 的拒绝性质相同, 其公式虽然列出两个同参与方、不同发送实例和不同消息的前置 Send, 却没有把这些三元组绑定到 Reject(bid, rst) 所对应的 Collected。因此, 机器结果的最强表述仍是“拒绝分支非空”。规则层可以直接看出 RejectRepeatedParty 匹配参与方相同的已收集条目, 但现有 existence lemma 不是特定被拒条目的机器见证。

P 仅给出直接维护目标关系的一种抽象实现。真实系统可以由调用方、批次构造器、接收方接纳层或其他具有可靠参与方归属信息的组件维持该关系; 当前结果不规定唯一检查位置, 也不证明某个部署应当新增相同规则。

3.4 综合结果与性质关系分析

表 3 汇总三个模型源文件中的全部 14 个 model-lemma 实例。独立复核直接解析并证明三个未修改模型, 复核结果与历史记录逐项比较; Steps 仅用于对应本次运行记录, 不作为工具性能指标。

表 3 三种批处理接纳模型的形式化验证及独立复核结果

Table 3 Formal Verification and Independent Rerun Results for the Three Batch-Admission Models

Model	Lemma	Property Type	Outcome	Steps	Independent Rerun Match
R	normal_relaxed_batch_exists	exists-trace / 基本可达性	VERIFIED	10	MATCH
R	one_send_two_accepts_exists	exists-trace / 精确来源重复接受见证	VERIFIED	13	MATCH
R	receiver_accept_has_send	all-traces / 来源对应性	VERIFIED	8	MATCH
R	receiver_accept_injective	all-traces / 作用域内精确来源单射性	FALSIFIED WITH COUNTEREXAMPLE	13	MATCH
M	repeated_message_rejection_exists	exists-trace / 拒绝分支可达性	VERIFIED	4	MATCH
M	same_party_different_messages_batch_exists	exists-trace / 同参与方不同消息见证	VERIFIED	16	MATCH
M	accepted_batch_has_distinct_messages	all-traces / 消息区分	VERIFIED	31	MATCH
M	receiver_accept_has_send	all-traces / 来源对应性	VERIFIED	8	MATCH
M	receiver_accept_injective	all-traces / 作用域内精确来源单射性	VERIFIED	33	MATCH
P	same_party_rejection_exists	exists-trace / 拒绝分支可达性	VERIFIED	5	MATCH
P	distinct_party_batch_exists	exists-trace / 不同参与方双接受见证	VERIFIED	17	MATCH
P	accepted_batch_has_distinct_parties	all-traces / 参与方区分	VERIFIED	31	MATCH
P	receiver_accept_has_send	all-traces / 来源对应性	VERIFIED	8	MATCH
P	receiver_accept_injective	all-traces / 作用域内精确来源单射性	VERIFIED	33	MATCH

独立复核环境为 WSL Ubuntu 24.04、Tamarin 1.12.0 和

Maude 3.5.1。manifest 记录的三个模型 SHA-256 与当前权威

源文件一致；7 次 version/parse/prove 调用均正常结束，三个 prove transcript 的 wellformedness checks 均成功。14 项状态和步数全部匹配，其中 13 项为 verified，1 项为 falsified with trace；证据包保留 raw stdout/stderr 以及 7 个 JSON graph。该证据是 2026 年 9 月 16 日针对明确代码版本完成的独立重跑，不是历史 freeze 日志的恢复。它确认当前抽象结果可重复，但不产生新的安全性质，也不补足从 K-Waay 到抽象模型的精化关系。

为比较三个坐标，沿用第 2.4 节在一条当前共同生命周期轨迹 τ 上定义的参与方区分 $P\tau$ 、消息区分 $M\tau$ 和作用域内精确来源单射性 $I\tau$ 。在消息由每次 SendMessage 新鲜生成、每个接受具有更早的精确匹配来源、且事件固定在同一 (bid, rst) 的条件下，有人工推导关系

$$(P\tau \Rightarrow M\tau) \wedge (M\tau \Leftrightarrow I\tau). \quad (25)$$

各方向的理由如下。若 $M\tau$ 成立，同一 (A, oid, m) 在两个不同时间点被接受会直接违反消息区分，因此 $M\tau \Rightarrow I\tau$ 。反之，若两个不同接受事件具有相同消息，则来源对应性和 fresh m 迫使二者追溯到同一个 Send(A, oid, m)； $I\tau$ 随即排除两个不同接受时间点，故 $I\tau \Rightarrow M\tau$ 。同样地，相同消息迫使精确来源及参与方坐标相同，因此 $P\tau$ 排除这种情况，得到 $P\tau \Rightarrow M\tau$ 。这些方向依赖当前 freshness、origin matching 和事件作用域，是规则层人工推导，不是 Tamarin 单独验证的跨理论 lemma。

M 的机器见证进一步给出

$$M\tau \not\Rightarrow P\tau. \quad (26)$$

accepted_batch_has_distinct_messages 说明 M 的全部轨迹满足消息区分，same_party_different_messages_batch_exists 则给出一条消息不同而参与方相同的双接受轨迹；二者共同支持这一非蕴含判断。这里仍需区分三个证据层次：具体 theory 的 all-traces lemma 约束该 theory 的全部轨迹，exists-trace lemma 只给出一条见证，而 P/M/I 的前三个蕴含方向是基于共同规则的人工逻辑推导。

综合来看，单个条目的来源对应性、接受消息之间的区分以及同一批次内的参与方区分是三个不同研究对象。当前模型中的消息级限制能够排除精确消息重复和相应的重复接受形状，却不能替代 K-Waay 接口所要求的批内参与方关系；参与方级模型则为该目标关系提供安全性与非空性的正控制。上述归纳止于固定两槽、理想精确来源和 ReceiverAccept 观察边界，不能升级为完整 K-Waay 协议的安全失效、密钥后果或任意批次大小结论。

4 讨论与设计启示

4.1 受控比较的语义

取消 $A_1 \neq A_2$ 后，变量可以取相同值，这一语法现象本身并不能充分说明原条件的作用。本文的受控比较关注的是：在共同的发送、收集、批次创建和接收处理生命周期中，若一次批处理接纳仍被解释为汇集不同参与方的输入，需要维持哪一层关系。R、M、P 三个模型分别放宽所有坐标限制、只比较消息坐标以及直接比较参与方坐标，因此能够在相同观察终点上区分精确输入重复、同参与方不同消息和不同参与方正常批次。

R 中的见证不是一个孤立的变量赋值。一个具有精确匹配来源的公开条目先后进入两个槽位，经批次接纳和顺序处理后，在同一 (bid, rst) 中产生两个参数相同、时间点不同的 ReceiverAccept。这一结果把接纳条件的放宽与模型接收事件的多重出现联系起来，但观察终点仍是符号模型事件，不是应用动作或密钥安装。

M 提供了区分参与方关系与精确重复的关键对照。它排除了同一消息坐标占据两个已接受位置，并满足作用域内精确来

源单射性；然而，同一参与方通过两个不同发送实例产生的不同消息仍可在一个批次中分别被接受。两个接受各自具有不同的精确发送来源，所以该见证并不依赖来源错配。由此可见，防止一个精确来源被重复接受，并未回答批次是否由不同参与方组成。P 则直接维护目标参与方关系，并由不同参与方双接受见证排除安全性质的空真解释。这组比较是针对 K-Waay 已声明条件的协议特定分析，不构成新的通用验证方法。

4.2 参与方、消息与来源单射性的关系

参与方身份、消息身份和事件出现次数回答不同问题。参与方坐标 A 表示一个条目投影到哪个模型主体；消息坐标 m 用于比较两个条目的符号消息是否相同；作用域内精确来源单射性 $I\tau$ 则约束同一 (bid, rst) 中，一个匹配的 Send(A, oid, m) 是否能支持两个不同时间点的匹配接受事件。最后一种关系讨论事件重数，并不要求不同消息属于不同主体。

同一参与方 A 可以产生 $oid_1 \neq oid_2$ 、 $m_1 \neq m_2$ 的两个条目。各条目只被接受一次时，精确来源没有重复，消息也彼此不同，但它们在同一批次中的参与方投影仍相同。第 3.4 节给出的

$$(P\tau \Rightarrow M\tau) \wedge (M\tau \Leftrightarrow I\tau), \quad M\tau \not\Rightarrow P\tau \quad (27)$$

依赖当前模型的新鲜消息、精确来源对应和事件作用域。它不是把三个性质排列成逐级增强的一般层次，也没有证明它们在任意协议中的全部逻辑独立关系。这里引入 $I\tau$ 的作用是区分“一个精确来源被多次接受”与“一个参与方具有多个独立来源”；研究目标始终是批内参与方组合关系。

4.3 语义不变量与维护责任

DistinctPartyPerBatch 描述一次已接纳批次中的条目应满足什么关系，而维护机制说明由哪个组件以及以何种方式保持该关系。P 的接纳规则直接比较 A_1 与 A_2 ，为符号抽象提供了一种可执行的正控制；这一实现不意味着真实系统只能在接收方使用同样的检查。

若系统确实要求一个批次中的位置代表不同参与方，可信调用方可以只构造参与方不同的批次，批次构造器可以在组合过程中维护该关系，接收方接纳层也可以在处理前检查它。其他能够获得可靠参与方归属的组件同样可能承担这一责任。这些位置只是体系结构上的可能性，本文没有评估具体部署，也没有证明某个 K-Waay 实现遗漏了检查。

维护条件所依据的表示也需要明确。模型中的 A 是协议主体坐标，并不自动等同于账户名、公钥编码、用户名、预密钥记录或数据库行。具体系统只有在建立从这些表示到参与方坐标的合理映射后，相关比较才具有本文所分析的语义。当前模型既没有验证该映射，也没有验证提供参与方归属信息的组件是否可信。

4.4 与认证、重放和去重的关系

条目级来源关系与批次内部的参与方关系是两项不同义务。当前模型中的来源对应只表示一个已接受三元组之前存在匹配的 Send(A, oid, m)，不等于完整 K-Waay 认证。即使两个条目各自具有合法模型来源，也可能像 M 的见证一样共同重复参与方。因此，单个条目的来源有效性不能自动推出多个条目的组合有效性。

R 的见证具有精确输入重放的解释，因为同一个公开三元组被送入两个位置；但参与方问题并不只由这一形状产生。M 的见证使用不同消息和不同发送来源，不依赖对一条精确消息的重复。排除 R 中的形状后，仍需单独判断同一参与方的多个独立条目能否进入一个批次。

消息去重在 M 所编码的目标上是有效的：相等消息的抽象拒绝分支可达，已接受消息彼此区分，作用域内精确来源单射性也成立。它不能推出参与方区分，并不表示去重或重放防护对自身目标无效，而是说明被比较的坐标与目标关系不同。本文的比较使用新鲜符号消息，不评估具体报文格式上的去重

算法,也不分析跨批次重放缓存。

4.5 接口设计启示

首先,应把组合过程视为具有独立语义条件的接口边界。多个条目分别满足来源要求,不代表它们的联合排列自然满足批次要求;接口规范需要同时说明单个输入条件与批内参与方关系。

其次,限制应作用于目标解释所依据的坐标。如果期望的是不同参与方,消息、发送实例或槽位之间的不等关系不能仅因它们都能区分条目而替代参与方不等。若要使用另一种表示或比较方式,需要另行说明它与目标参与方关系之间的联系。

最后,应在集成契约中明确由哪个组件维持该关系,以及该组件依赖何种参与方归属信息。这一启示不要求为 K-Waay 增加某个特定修补规则;原规范已经声明不同参与方条件,本文只说明在当前抽象下保持其批次语义所需的关系。任何从重接受继续推导到应用状态的结论,都需要显式的消费者模型。

4.6 分析边界

固定两槽。DistinctPartyPerBatch 的语义定义允许任意有限批次,但机器证据固定 $n = 2$ 。两槽表示最小违反对,不能证明更大批次或任意 n 的性质,也不提供跨批次、回滚或重启保证。

接纳层抽象。模型只表示组合、接纳和来源匹配的接收事件,没有编码签名、KEM、KDF、预密钥密码运算、ratchet 或攻陷行为。新鲜发送实例、消息坐标以及持久来源事实都是原型选择;源接口与模型之间没有 refinement 定理。攻击者能够控制候选条目的组成是分析假设,不是针对真实批次构造器已经证实的能力。

接收事件终点。直接证据止于 ReceiverAccept。模型没有输出密钥、KEY/TEST 接口或应用消费者,因而重复接受本身不能证明密钥接口失败、重复安装、状态破坏或应用受损。

拒绝性质。M/P 的拒绝 lemma 只证明某条抽象拒绝轨迹可达,不证明所有无效候选批次最终都会拒绝。same_party_rejection_exists 也没有把前置的两个 Send 三元组绑定到实际被拒的 Collected 条目。规则语义能够解释同参与方条目的拒绝分支,但现有 lemma 不是针对某个具体被拒输入的机器见证;模型也没有规定真实接口是丢弃单项、拒绝整批还是返回其他错误。

表示与执行位置。模型把参与方坐标视为显式符号值,没有验证身份解析、公钥到参与方的映射、调用方执行或分布式维护。第 4.3 节列举的执行位置是设计选择,不是已验证实现结论。

复核来源。英文母稿撰写时记录的“缺少原始 prover transcript、尚未独立重跑”已经被后续证据更新。2026 年 9 月 16 日的独立复核在 Tamarin 1.12.0、Maude 3.5.1 环境中直接解析并证明三个未修改模型,14 项状态和步数与历史记录全部匹配,并保留原始标准输出、错误输出和 JSON graph。该复核提高了当前抽象结果的可重复性,但不是历史日志的恢复,也没有扩展模型范围或建立源协议到模型的精化关系。

5 相关工作

5.1 K-Waay 与后量子异步密钥交换

K-Waay 是本文所分析接口的直接来源。Collins 等利用 split-KEM 构造具有 X3DH 类异步交互方式的后量子可否认认证密钥交换,提出在接收侧复用临时材料的 BatchReceive,并给出相应的计算安全定义与证明[4]。完整规范同时明确要求,一次 BatchReceive 调用中的元素对应不同参与方(见文献[5], Sec. 5.1, p. 21)。因此,批内不同参与方条件属于既有规范;本文不把它作为新发现,也不提出针对“遗漏检查”的修补,而是隔离批处理接纳边界,分析该条件维护的参与方关系。

异步安全消息协议的形式化研究提供了更广的背景。Signal 分析把 X3DH 与 Double Ratchet 建模为多阶段认证密钥交换系统,并证明特定范围内的密码会话性质[1]。Bhargavan 等将 PQXDH 规范转换为 ProVerif 和 CryptoVerif 模型,分析认证、机密性、后量子前向保密及共存部署,并澄清规范假设[3]。X3DH 规范也单独讨论认证、重放、密钥复用和身份绑定[9]。这些工作说明精确建模身份和会话的重要性,但主要对象是双方握手、ratchet 会话和密码密钥安全。本文不重新验证上述构造,而只研究一个批次内部的参与方关系,观察终点为符号接收事件。

5.2 安全消息组合的形式化分析

已有研究表明,组合层可能具有组件性质不能直接推出的安全义务。Cremers、Jacomme 和 Naska 分析管理多个 ratchet 会话并将其合并为对话的应用层,说明对话层的攻陷后安全性需要与单条 ratchet 链的安全性分开研究[2]。这一工作为把安全消息原语之上的接口作为独立形式化对象提供了最接近的方法先例,但其组合对象是会话到对话的管理,而本文考察一次接收批次中多个条目的参与方投影。

群组消息研究进一步展示了携带身份信息的组合状态,但没有定义本文的批内不变量。TreeSync 对 MLS 认证群组管理状态及其一致性、完整性给出可执行形式化描述[10];cryptographic administration 研究群组更新中的授权和成员一致性[11];MLS 的模块化分析从明确分离的密码组件推导安全群组消息性质[12];跨群组分析则说明恢复性质取决于用户和攻陷影响在多个群组间的表示[13]。这些工作分别关注演化的群组状态、成员授权、密码组件组合或跨群组攻陷,均不等同于要求一个 BatchReceive 批次的两个位置投影到不同参与方。

安全消息研究的系统分类也区分参与者一致性、发言者一致性、目的地、认证及消息级性质[14]。其中参与者一致性关注诚实方是否对参与者列表形成一致认识,本文谓词则比较一个批次内部不同位置的参与方投影,二者不是同一关系。一般组合验证框架研究协议组合后组件性质得以保持的条件[15],说明组合层推理本身早已有之。本文采用 Tamarin 的多集重写轨迹框架同样属于既有方法[8];工具的使用不会自动把固定两槽抽象扩展到任意批次。本文的范围只是在特定 K-Waay 接口上分析同批参与方区分这一组合对象和性质。

5.3 身份语义与错误绑定

主体名称及其身份解释的安全意义已有系统研究。Lupetti、Dillema 和 Stabell-Kulø 指出,名称需要在适当的关联会话范围内保持唯一,才能保留预期的协议含义,并讨论局部和全局验证方式[6]。chosen-name 分析从另一角度说明,对手选择或分配代理名称的能力会改变形式化攻击面[16]。因此,身份唯一性以及身份表示应作为建模决策,并不是本文提出的一般新观点。

本文使用的关系更窄:在已经给出显式参与方坐标的前提下,判断同一坐标能否占据一个已接纳批次的两个槽位。模型不分析两个主体是否共享有歧义的名称,也不研究对手合成参与方标识。区分参与方、消息、发送实例和槽位,是把既有身份语义应用到 BatchReceive 的协议特定问题,不构成新的身份理论。

Misbinding 与未知密钥共享工作在概念上相邻,但失败定义不同。形式化 misbinding 分析考察端点、设备、用户或协议运行与错误身份关联的情形[17];TLS/SDP 的未知密钥共享分析则涉及把外部身份绑定到另一端点的密钥、指纹或会话[18]。本文的重复参与方组合是同一个模型参与方坐标出现两次,当前模型既没有证明错误的端点—身份关联,也没有证明双方对共享密钥的伙伴认识不一致。因此,R/M 的见证不能归类为 misbinding、设备配对攻击或未知密钥共享攻击。

5.4 单射性、重放与坐标级控制

Injective agreement 是既有认证概念。Lowe 的认证层次区分 aliveness、weak agreement、non-injective agreement 和 injective agreement，其中单射性要求相关协议运行之间形成一一对应[7]。Sattarzadeh 和 Fallah 给出 injective agreement 的自动类型化处理，使用唯一匹配的 begin/end 资源排除由一条发送消息支持多个接收事件的情况[19]。X3DH 规范同样明确讨论重放和重复接受问题[9]。单射对应和重放敏感性质都不是本文首次提出。

本文只把作用域内来源单射性作为诊断性对照。在一个批次和接收方上下文内，M 允许每个精确发送来源最多支持一次接受，同时仍允许同一参与方的两个不同消息分别被接受。该性质不保证每个 Send 最终交付，也不排除跨批次复用；其作用是说明事件出现次数与参与方区分刻画不同关系，而不是否定既有 injectivity 概念。相同地，精确消息去重处理的是消息坐标相等。受控模型只支持“该限制在当前生命周期下不蕴含参与方级目标”这一较窄结论，不能推出重放防护或去重对自身目标无效。

5.5 本文工作的定位

与本文最接近的既有工作分别提供了 K-Waay 的原始接口条件[5]、安全消息组合层的研究先例[2]、作用域内名称唯一性的既有语义[6]以及事件级单射性的概念工具[19]。它们的分析对象均不同于本文针对 K-Waay 批处理接纳的受控比较，因而本文不主张身份或组合推理上的一般优先性。

在既定边界内，当前分析有两项直接证据。其一，消息级对照显示：消息区分和作用域内来源单射性成立时，同参与方不同消息的组合仍然可达。其二，这一分离位于 R 的精确来源重复接受见证和 P 的非空参与方级正控制之间。两项结论都只针对固定两槽接纳抽象及其已记录性质，不适用于任意批次长度或完整 K-Waay 执行。

消息与身份的区别、名称绑定的重要性、单射对应、组合层推理以及语义不变量与执行机制的区分都不是本文的一般理论贡献。本文的工作是把这些已有区分应用于特定 BatchReceive 参与方接纳问题，并比较消息坐标限制与直接维护参与方关系的不同结果。该定位不声称原协议遗漏其已声明条件、计算安全定理无效或部署实现违反不变量，也不从有界接收事件证据推出密码学破坏、misbinding、未知密钥共享、应用后果或任意批次定理。

6 结束语

针对 K-Waay 一次 BatchReceive 调用中不同输入对应不同参与方的既有条件，构建了共享生命周期的 R、M、P 三种固定两槽 Tamarin 接纳模型。R 的机器见证表明，放宽坐标限制后，一个精确发送来源可以在同一批次和接收方上下文中支持两个不同时间点的 ReceiverAccept，相应的作用域内精确来源单射性被反例否定。M 排除了这一精确重复形状，并验证了消息区分和来源单射性，但同一参与方通过两个独立发送实例产生的不同消息仍可共同被接受。因此，在当前编码的生命周期和来源假设下，消息区分及事件单射性不能替代批内参与方区分。

P 从参与方坐标上直接维护目标关系；已验证的安全性质说明同一上下文中的接受事件具有不同参与方，distinct_party_batch_exists 同时给出有效双接受执行，排除了性质仅因处理路径不可达而成立的解释。由此得到的有限设计启示是：批处理接口应明确其输入所需的参与方关系，并指定负责维护该关系的组件，而不能从消息级检查推定参与方级组合条件。独立复核确认三个未修改模型的 14 项结果与历史记录一致，但证据仍止于理想精确来源匹配下的固定两槽 ReceiverAccept；将其扩展到具体 K-Waay 密钥输出、任意批

次或真实部署行为，需要额外模型与精化依据。

参考文献

- [1] Katriel Cohn-Gordon, Cas Cremers, Benjamin Dowling, et al. A Formal Security Analysis of the Signal Messaging Protocol[J]. Journal of Cryptology, 2020, 33: 1914-1983.
- [2] Cas Cremers, Charlie Jacomme, Aurora Naska. Formal Analysis of Session-Handling in Secure Messaging: Lifting Security from Sessions to Conversations[C]/32nd USENIX Security Symposium, 2023: 1235-1252.
- [3] Karthikeyan Bhargavan, Charlie Jacomme, Franziskus Kiefer, et al. Formal Verification of the PQXDH Post-Quantum Key Agreement Protocol for End-to-End Secure Messaging[C]/33rd USENIX Security Symposium, 2024: 469-486.
- [4] Daniel Collins, Loïs Huguenin-Dumittan, Ngoc Khanh Nguyen, et al. K-Waay: Fast and Deniable Post-Quantum X3DH without Ring Signatures[C]/33rd USENIX Security Symposium, 2024: 433-450.
- [5] Daniel Collins, Loïs Huguenin-Dumittan, Ngoc Khanh Nguyen, et al. K-Waay: Fast and Deniable Post-Quantum X3DH without Ring Signatures[EB/OL]. Cryptology ePrint Archive, Paper 2024/120, 2024[2026-10-06]. <https://eprint.iacr.org/2024/120>.
- [6] Simone Lupetti, Feike W. Dillema, Tage Stabell-Kulø. Names in Cryptographic Protocols[C]/4th International Workshop on Security in Information Systems (WOSIS 2006), 2006: 185-194.
- [7] Gavin Lowe. A Hierarchy of Authentication Specifications[C]/10th IEEE Computer Security Foundations Workshop, 1997: 31-43.
- [8] Simon Meier, Benedikt Schmidt, Cas Cremers, et al. The TAMARIN Prover for the Symbolic Analysis of Security Protocols[C]/CAV 2013; LNCS 8044, 2013: 696-701.
- [9] Moxie Marlinspike, Trevor Perrin. The X3DH Key Agreement Protocol[EB/OL]. Signal protocol specification; Revision 1, 2016[2026-10-06]. <https://signal.org/docs/specifications/x3dh/>.
- [10] Théophile Wallez, Jonathan Protzenko, Benjamin Beurdouche, et al. TreeSync: Authenticated Group Management for Messaging Layer Security[C]/32nd USENIX Security Symposium, 2023: 1217-1233.
- [11] David Balbás, Daniel Collins, Serge Vaudenay. Cryptographic Administration for Secure Group Messaging[C]/32nd USENIX Security Symposium, 2023: 1253-1270.
- [12] Joël Alwen, Sandro Coretti, Yevgeniy Dodis, et al. Modular Design of Secure Group Messaging Protocols and the Security of MLS[C]/ACM CCS 2021, 2021: 1463-1483.
- [13] Cas Cremers, Britta Hale, Konrad Kohbrok. The Complexities of Healing in Secure Group Messaging: Why Cross-Group Effects Matter[C]/30th USENIX Security Symposium, 2021: 1847-1864.
- [14] Nik Unger, Sergej Dechand, Joseph Bonneau, et al. SoK: Secure Messaging[C]/2015 IEEE Symposium on Security and Privacy, 2015: 232-249.
- [15] Suzana Andova, Cas Cremers, Kristian Gjøsteen, et al. A Framework for Compositional Verification of Security Protocols[J]. Information and Computation, 2008, 206(2-4): 425-459.
- [16] Pieter Ceelen, Sjouke Mauw, Saša Radomirovic. Chosen-Name Attacks: An Overlooked Class of Type-Flaw Attacks[J]. Electronic Notes in Theoretical Computer Science, 2008, 197(2): 31-43.
- [17] Aleksi Peltonen, Mohit Sethi, Tuomas Aura. Formal Verification of Misbinding Attacks on Secure Device Pairing and Bootstrapping[J]. Journal of Information Security and Applications, 2020, 51: 102461.
- [18] Martin Thomson, Eric Rescorla. Unknown Key-Share Attacks on Uses of TLS with the Session Description Protocol (SDP)[R/OL]. RFC Editor, RFC 8844, 2021[2026-10-06]. <https://www.rfc-editor.org/rfc/rfc8844.html>.
- [19] B. Sattarzadeh, M. S. Fallah. Automated Type-Based Analysis of Injective Agreement in the Presence of Compromised Principals[J]. Journal of Logical and Algebraic Methods in Programming, 2015, 84(5): 576-610.